Southeast University

Report on Principles of Compiler

Title XL-Scanner

School of Software School (Department) software engineering Major

Student ID 71122135

Student Name Pengyu Xie

Design Location Nanjing

Table

Chapter 1 Lexical Analyzer Implementation

1.1 Aim

The compilation process can be divided into five main stages: lexical analysis, syntax analysis, intermediate code generation, code optimization, and target code generation. Among these, a word is the smallest grammatical unit with actual meaning in high-level languages, and lexical analysis serves as the first structural layer of a compiler. A lexical analyzer scans the source program string, identifies valid words based on lexical rules, and converts them into a standardized format for further use by the syntax analyzer.

In this experiment, regular expressions are constructed to define lexical rules, and a DFA (Deterministic Finite Automaton) is employed to perform lexical analysis. This practical approach deepens the understanding and application of automata theory and lexical analysis principles. Additionally, it enhances programming and algorithm design skills, laying a solid foundation for compiler-related technologies.

1.2 Content description

1) Define REs by yourself for the common tokens used in our programming languages.

2) Convert REs into NFAs.

3) Merge these NFAs into a single NFA.

4) Convert the NFA into a DFA with minimum states.

5) Develop a scanner based on the DFA.

1.3 Methods

In this project, regular expressions (REs) are defined to represent common tokens found in programming languages. These tokens include identifiers, keywords, operators, and delimiters. The regular expressions will act as formal patterns to describe the syntax of these tokens precisely.

The defined regular expressions are then systematically converted into nondeterministic finite automata (NFAs). This process involves constructing state transition diagrams for each RE, illustrating their acceptance of valid tokens.

To handle multiple token types within a single framework, the NFAs derived from individual REs are merged into one unified NFA. This combined NFA can efficiently represent all token definitions simultaneously, preparing for further deterministic analysis.

The unified NFA is then converted into a deterministic finite automaton (DFA) to ensure unambiguous and efficient state transitions during token recognition. State minimization is applied to the DFA to reduce complexity and optimize performance.

Finally, the minimized DFA is used as the foundation for developing a scanner. The scanner performs lexical analysis by traversing the DFA, recognizing tokens, and outputting them for further processing in a compiler or interpreter.

This step-by-step approach integrates both theoretical and practical aspects of lexical analysis, offering a comprehensive framework for understanding and implementing a scanner.

1.4 Assumptions

In this experiment, I selected a subset of C++ syntax as the input object for lexical analysis. These input objects include common components such as identifiers, keywords, operators, delimiters, and constants. By categorizing these words, we can clearly define the types of tokens to be processed in the experiment and the corresponding lexical rules, providing a clear objective for implementing the lexical analyzer. Below is the table of word classifications used in this experiment, along with detailed explanations.

Token Type	Token Name	Lexeme
Keywords	IF	if
	THEN	then
	ELSE	else
	READ	read
	WRITE	write
	WHILE	while
	FOR	for
	RETURN	return
	SWITCH	switch
	CONTINUE	continue
	CASE	case
	VOID	void
	INT	int
	DOUBLE	double

Table 1.1 Lexical Unit Classification Table (Part 1)

Token Type	Token Name	Lexeme
Keywords	CIN	cin
	COUT	cout
	ENDL	endl
Operators	PLUS	+
	MINUS	-
	MULTIPLE	*
	DIVIDE	/
	EQUAL	=
	BIGGER	>
	SMALLER	<
	E_COMPARE	= =
	BIGGER_EQUAL	>=
	SMALLER_EQUAL	<=
	NOT_EQUAL	!=
	AND	&&
	OR	||
	NOT	!
	SELF_PLUS	++
	SELF_MINUS	- -
	LEFT_SHIFT	< <
	RIGHT_SHIFT	> >
Delimiters	LEFT_YKH	(
	RIGHT_YKH	)
	LEFT_FKH	[
	RIGHT_FKH	]
	LEFT_DKH	{
	RIGHT_DKH	}
	COMMA	,
	SEMOCOLOM	;
	COLON	:
	QUOTATION1	’
	QUOTATION2	”
	ZS	#

Table 1.2 Lexical Unit Classification Table (Part 2)

Token Type	Token Name	Lexeme
Identifiers	ID	Identifier
Numbers	INT	Integer
	DOUBLE	Floating-point

Table 1.3 Lexical Unit Classification Table (Part 3)

The lexical analysis program is based on finite state automaton programming. First, define the lexical regular expression as follows:

$$
\begin{aligned}
S &\rightarrow K \mid O \mid F \mid \text{letter}A \mid \text{digit}B \\
K &\rightarrow \text{if} \mid \text{then} \mid \ \ldots \ \mid \text{case} \\
F &\rightarrow (\mid) \mid \ \ldots \ \mid \# \\
A &\rightarrow \text{letter}A \mid \text{digit}A \mid \epsilon \\
B &\rightarrow \text{digit}B \mid .\text{digit}C \mid \epsilon \\
C &\rightarrow \text{digit}C \mid \epsilon \\
\text{letter} &\rightarrow a \mid b \mid \ \ldots \ \mid z \\
\text{digit} &\rightarrow 0 \mid 1 \mid \ \ldots \ \mid 9
\end{aligned}
$$

In the given grammar, S is the start symbol, representing the beginning of a sentence. The set of non-terminal symbols is $V_N = \{K, O, F, C, letter, digit\}$, where:

- K represents **keywords**, such as `if`, `else`, `while`, etc.;
- O represents **operators**, such as `+`, `-`, `*`, `/`, etc.;
- F represents **delimiters**, such as `(`, `)`, `,`, `;`, etc.;
- C represents **components of composite identifiers or numbers**;
- $letter$ represents the set of **alphabetic characters**, including $a, b, \ldots, z$;
- $digit$ represents the set of **numeric characters**, including $0, 1, \ldots, 9$.

The set of terminal symbols V_T defines all possible concrete symbols in the grammar. The terminal symbol set is defined as:

$$V_T = \{\text{Keywords, Operators, Delimiters, Alphabetic Characters, Numeric Characters}\}$$

where:

- **Keywords**: such as `if`, `else`, `for`, `while`, etc.;
- **Operators**: such as `+`, `-`, `*`, `/`, etc.;

- **Delimiters**: such as `(`, `)`, `{`, `}`, `;`, etc.;
- **Alphabetic Characters**: such as $a, b, c, \ldots, z$;
- **Numeric Characters**: such as $0, 1, 2, \ldots, 9$.

Through the above definitions, the non-terminal symbols and terminal symbols in the grammar clearly distinguish between abstract structures and concrete symbols, facilitating further analysis and application.

1.5 Related FA descriptions

1.5.1 NFA

Based on the given regular expressions, a corresponding non-deterministic finite automaton (NFA) can be constructed for each regular expression. These individual NFA machines describe the matching process for specific lexical units, such as keywords, operators, delimiters, etc. By merging these independent NFAs into a unified NFA, a complete state machine can be built to recognize all possible input character patterns.

The merged NFA is constructed by adding a new initial state and introducing ε-transitions from this initial state to the starting state of each independent NFA. This design ensures that each regular expression's corresponding state machine remains independent while supporting global matching through the merging process.

The following figure 1.1 illustrates the merged NFA, which integrates the functionality to recognize keywords, operators, delimiters, identifiers, and numeric values.

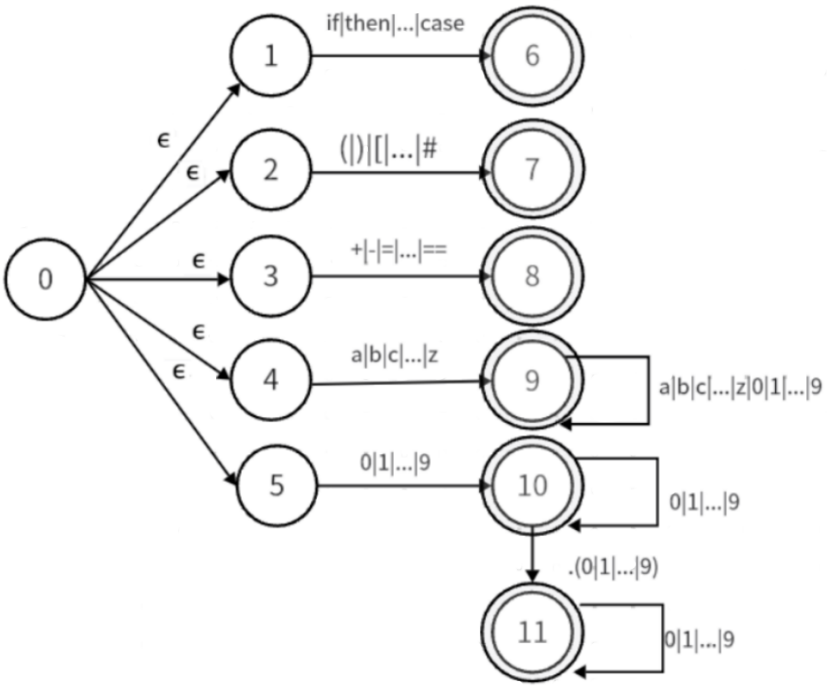

Figure 1.1 NFA for Lexical Analysis

1.5.2 DFA

Next, the subset construction method is used to convert the NFA into an equivalent DFA (Deterministic Finite Automaton). The core idea of the subset construction method is to map a set of NFA states to a single DFA state, thereby eliminating the non-determinism in the NFA. This ensures that the DFA has a finite number of states and uniquely determines the next state for each input symbol.

The first step of the conversion process is to read the initial state of the NFA, $I_0 = \{0, 1, 2, 3, 4, 5\}$. This initial state consists of multiple states from the NFA, representing the set of states the NFA can simultaneously be in at the start. Then, based on the NFA's state transition mapping function, the closure of the state set for each input symbol is computed to generate new DFA states.

By iterating through all input symbols and recursively constructing new state sets, a complete DFA transition table can be generated. The following table is the determinization table 1.4 derived from the subset construction method, illustrating the process of converting the NFA into a DFA.

State	Keyword	Operator	Delimiter	Letter	Digit	.digit
$I_0 = \{0, 1, 2, 3, 4, 5\}$	$I_1 = \{6\}$	ϵ	$I_3 = \{7\}$	$I_4 = \{9\}$	$I_5 = \{10\}$	ϵ
$I_1 = \{6\}$	ϵ	ϵ	ϵ	ϵ	ϵ	ϵ
$I_2 = \{8\}$	ϵ	ϵ	ϵ	ϵ	ϵ	ϵ
$I_3 = \{7\}$	ϵ	ϵ	ϵ	ϵ	ϵ	ϵ
$I_4 = \{9\}$	ϵ	ϵ	ϵ	$I_4 = \{9\}$	$I_5 = \{10\}$	ϵ
$I_5 = \{10\}$	ϵ	ϵ	ϵ	ϵ	$I_5 = \{10\}$	$I_6 = \{11\}$
$I_6 = \{11\}$	ϵ	ϵ	ϵ	ϵ	ϵ	ϵ

Table 1.4 DFA Determinization Table

Based on the state transition matrix derived from the table, the corresponding state transition diagram can be drawn. This diagram clearly illustrates the transition relationships between various states. The state transition diagram is the core result of DFA (Deterministic Finite Automaton) determinization, which eliminates the non-determinism of NFA (Non-deterministic Finite Automaton) by applying uniquely determined transition rules, thus enabling more efficient input symbol matching.

The state transition diagram provides an intuitive visualization of how each state transitions to a target state based on different input symbols. This information serves as an essential foundation for implementing a lexical analyzer or scanner using DFA. The following diagram 1.2 depicts the DFA state transition diagram after NFA determinization.

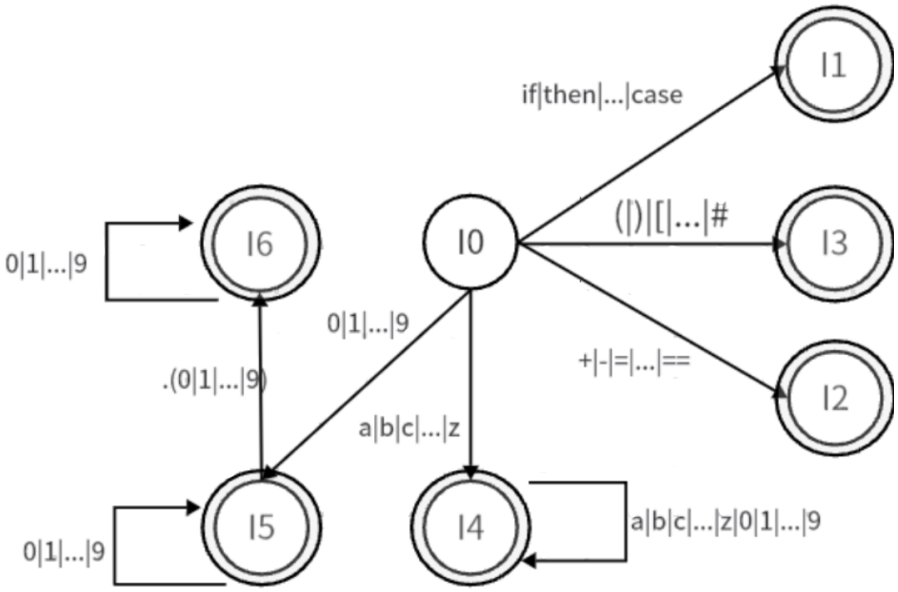

Figure 1.2 DFA for Lexical Analysis

1.5.3 mini DFA

It can be observed that this state machine is already the minimal state machine, having the smallest number of states. The minimized finite state machine not only significantly reduces the number of states but also simplifies the complexity of state transitions, thereby improving the efficiency of lexical analysis. Based on this minimal finite state machine, the design and implementation of the lexical analyzer will proceed.

1.6 Description of important Data Structures

In the lexical analyzer, appropriate data structures are designed to effectively classify and handle different types of lexical units:

Array:

The character categories are divided into the following seven types: **keyword**, **operator**, **delimiter**, **identifier**, **integer**, **decimal**, and **unrecognizable**. These categories are stored in an array for quick classification of input characters or strings. Among them, the **unrecognizable** type is mainly associated with the error detection function of the lexical analyzer. When encountering characters that do not match any predefined type, they are categorized as **unrecognizable** and trigger error handling in subsequent stages.The array implementation is shown in the following figure1.3.

```
/* 单词编码 */
char keyword[][20] = { "if","then", "read","write","case","continue","else","for","switch","while","
char delimiter[][10] = { "(",")","[","]","{","}",":",",",";","'","#",'"', };//12个界符
char operation[][10] = { "+","-","*","/","=",">","<","==",">=","<=","!=","&&","||","!","++","--","<<
```

Figure 1.3 Implementation of Array for Lexical Unit Categories

Enumeration Type:

The enumeration type is used to associate each lexeme (lexical unit) with its corresponding lexical unit type. Through the enumeration type, the type of a lexical unit can be precisely represented as a predefined enumeration value. Each lexeme (such as the keyword `if` or the operator `+`) is defined as a member of the enumeration type, facilitating matching and operations in the program.The implementation of the enumeration type is shown in the following figure1.4.

```
enum TokenCode
{
    // 关键字
    IF,
    THEN,
    READ,
    WRITE,
    CASE,
    CONTINUE,
    ELSE,
    FOR,
    SWITCH,
    WHILE,
    RETURN,
    K_VOID,
    K_INT,
    K_DOUBLE,
    K_CIN,
    K_COUT,
```

Figure 1.4 Implementation of Enumeration Types for Lexical Units

The design of these data structures supports the accuracy and efficiency of lexical analysis while providing a foundation for error detection and handling.

1.7 Description of core Algorithms

When receiving an input character stream, the lexical analyzer is responsible for reading characters one by one and recognizing lexemes, laying the foundation for subsequent syntax analysis and compilation. To ensure the lexical analyzer can function correctly, stably, and reliably, the following functional modules need to be designed and implemented:

1) **Lexical Unit Recognition Module**: Ensures that the lexical analyzer can accurately identify all lexical units in the subset of the programming language. These lexical units include **keywords** (e.g., `if` and `then`), **operators** (e.g., `+`, `-`, `*`, and `/`), **delimiters** (e.g., `()`, `{}`, and `;`), **identifiers**, and **numbers** (including integers and floating-point numbers). This module is the core functionality of the lexical analyzer and directly impacts its accuracy and reliability.

2) **Error Handling Module**: Used to process potential errors and non-conforming lexical units in the input text. For example, for unrecognized characters or improperly formatted inputs, the module needs to provide clear error messages and indicate the error's location, allowing users to quickly correct their code.

3) **Word Classification Module**: Responsible for correctly categorizing recognized lexical units, ensuring that each lexeme is accurately matched to its corresponding type (e.g., keywords, oper-

ators, identifiers, etc.). This module is critical for precise processing in subsequent compilation stages.

4) **Unified Format Conversion Module**: Ensures that the lexical analyzer can convert all recognized lexical units into a uniform internal representation. For instance, all identifiers can be converted into a standard type code, enabling the syntax analyzer to efficiently process these data.

5) **Result Output Module**: Responsible for outputting the results of lexical analysis in the form of a sequence of lexical unit tokens. This output serves as necessary input data for subsequent syntax analysis and other compilation phases. The output format needs to be standardized for convenient further processing.

Additionally, when processing large code files, the algorithm must guarantee sufficient efficiency and performance, ensuring that the lexical analyzer can complete the scanning and recognition of the input text within a reasonable time. Considering the need for future language extensions or modifications, the designed lexical analyzer should also possess a degree of flexibility and scalability to support the addition of new syntax rules or lexical unit types.

The algorithm flow chart is shown below as 1.5.

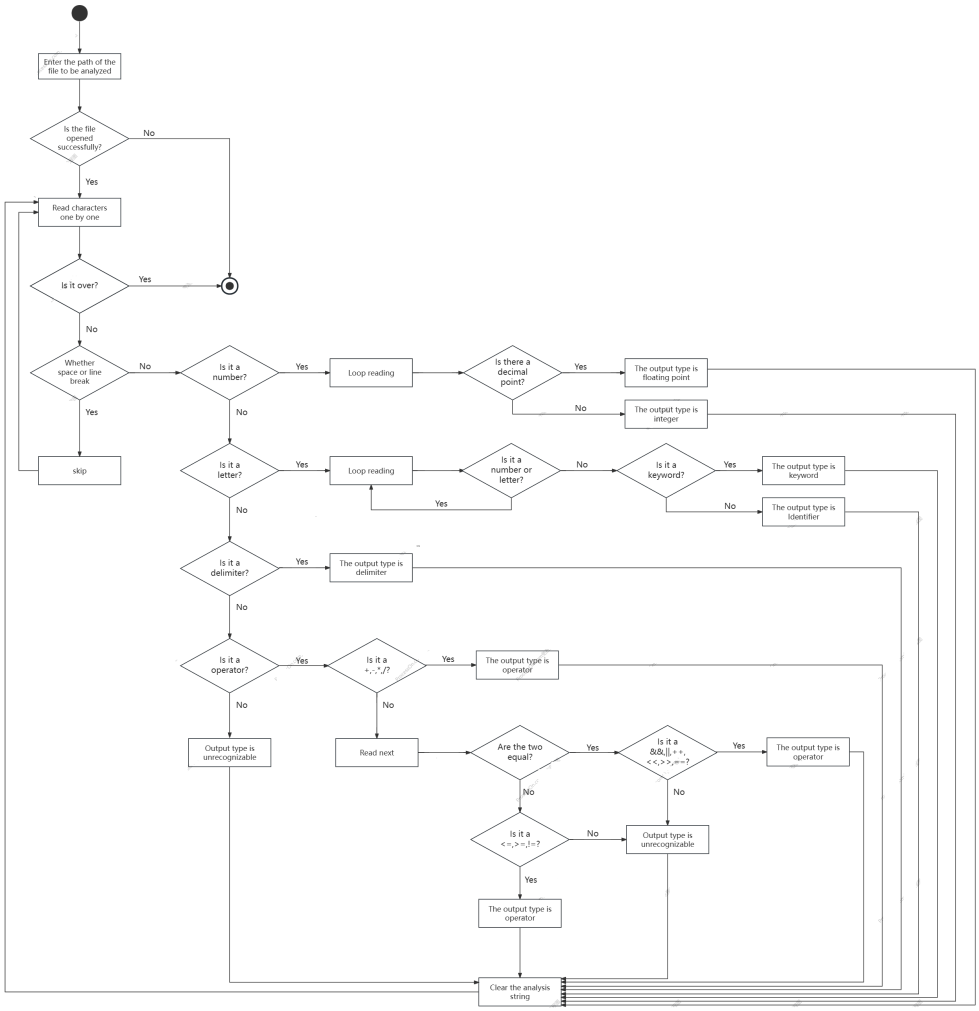

Figure 1.5 algorithm flow chart

When processing an input character stream, the lexical analyzer follows a structured workflow to recognize, classify, and handle each token in the source file. The process begins with the user entering the file path. If the file cannot be opened, an error message is displayed, and the program terminates. Otherwise, the analyzer initializes data structures and reads characters one by one.

The analyzer first checks if the end of the file has been reached. If so, it clears temporary data and terminates. If not, it skips whitespace characters like spaces or newlines. For numeric tokens, the analyzer checks if the character is a digit, enters a loop to read the full token, and determines whether it is an integer or a floating-point number by checking for a valid decimal point.

If the character is a letter, the analyzer assumes it could be a keyword or an identifier. It reads the complete token and compares it with a predefined keyword list. Matches are classified as keywords; otherwise, they are treated as identifiers. Delimiters, such as parentheses, braces, or semicolons, are directly matched against a list and classified accordingly.

For operators, simple characters like '+' or '-' are immediately classified, while potential double-character operators like '==' or '<=' require reading the next character to confirm validity. Unrecognized characters are flagged with error messages, indicating their position in the file.

After processing all characters, the analyzer cleans up temporary data and outputs a stream of tokens for the next compilation stage. This workflow ensures accuracy, efficiency, and robustness, providing a solid foundation for future extensions or language support.

1.8 Use cases on running

1.8.1 Correct example

Firstly, Figure 1.6 demonstrates the functionality of the designed lexical analyzer using a simple C++ code snippet. The logic of the code is clear and incorporates a variety of language components, including variable declarations, arithmetic operations, loop control structures, conditional statements, and delimiters, aiming to cover most of the defined lexical categories.

The result, shown in Figure 1.7, presents the output of the lexical analyzer after processing the aforementioned code. The lexical analyzer reads the character stream in the code one by one and classifies each part into different types of lexical tokens. For instance, the variable name `a` is identified as an identifier (`ID`), `=` is identified as an assignment operator (`EQUAL`), `0.0` is identified as a floating-point number (`T_DOUBLE`), and `for` is identified as a keyword (`FOR`). Additionally, arithmetic operators such as `+` and `*`, as well as delimiters like parentheses and semicolons, are correctly identified and categorized.

The results demonstrate that the lexical analyzer can accurately recognize all critical elements in the code and classify them into their corresponding lexical token types, effectively showcasing the core functionality of lexical analysis. The output is represented in the form of tuples, with each token containing its type and specific value. For example, `(ID, a)` represents the identifier `a`, and `(T_INT, 1)` represents the integer `1`. This structured output facilitates the subsequent stages (e.g., syntax analysis) and provides a reliable foundation for the entire compilation process.

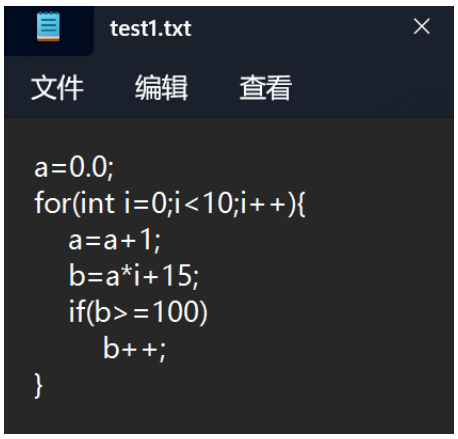

Figure 1.6 test1

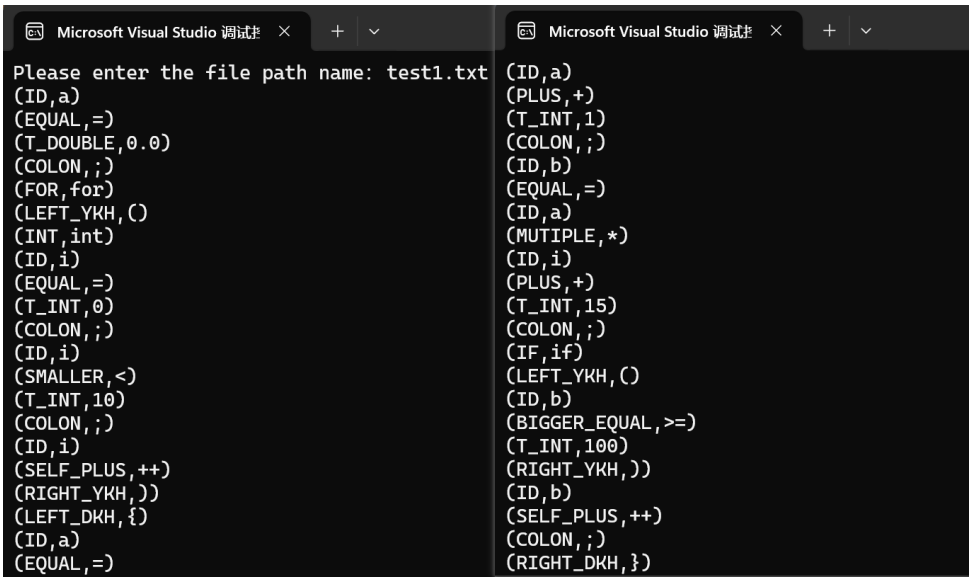

Figure 1.7 the result of test1

1.8.2 Floating point type error

In Figure 1.8, a C++ code snippet is presented to further test the lexical analyzer's capability in handling floating-point numbers and error detection. The snippet includes declarations for both an integer and a double variable, deliberately including an error in the second line to test the analyzer's robustness. Specifically, the floating-point number `1.11.3` contains multiple decimal points, which violates the definition of a valid floating-point number.

The result, shown in Figure 1.9, illustrates the output of the lexical analyzer. The analyzer correctly identifies the valid components of the first line, such as `int` being recognized as a keyword (`INT`), `i` as an identifier (`ID`), and `5.1` as a floating-point number (`T_DOUBLE`). Similarly, in the second line, the keyword `double` and the identifier `y` are correctly classified.

However, when processing the value `1.11.3`, the analyzer detects multiple decimal points in the number, which renders it invalid. According to the lexical rules, a number must either be an integer or contain at most one decimal point, with the decimal point not at the end of the number. Any violation of this rule results in an error message being displayed. In this case, the analyzer outputs a

`floating-point type error` for `1.11.` and terminates the analysis for the invalid token, demonstrating its ability to handle errors effectively.

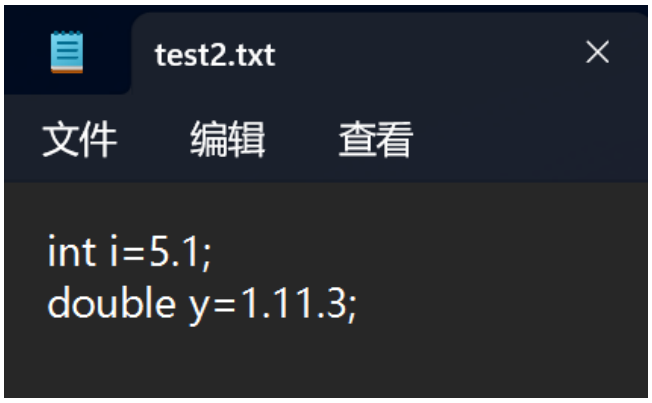

Figure 1.8 test2

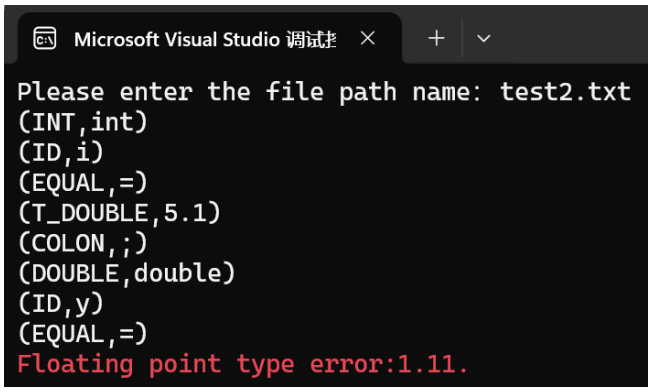

Figure 1.9 the result of test2

1.8.3 Unknown symbol error

In the test code (as shown in Figure 1.10), variable declarations, assignment operations, and numeric initializations are included. The numeric values comprise both integer and floating-point types. However, the second statement introduces an illegal character underscore (_), which is an undefined symbol in the lexical rules and, therefore, violates the valid lexical rules. When the lexical analyzer processes this code (as shown in Figure 1.12), it accurately categorizes and recognizes the valid portions. For instance, ‘int‘ is correctly recognized as a keyword (‘INT‘), ‘i‘ is identified as an identifier (‘ID‘), the number ‘5‘ is classified as an integer (‘T_INT‘), and the number ‘1.1‘ is recognized as a floating-point number (‘T_DOUBLE‘). Furthermore, delimiters such as ‘=‘, ‘;‘, etc., are also correctly classified.

However, when the lexical analyzer encounters the illegal character underscore (_), since it is not defined in the lexical rules, the analyzer cannot classify or process it. Consequently, the program immediately throws an error message: “Unrecognizable symbols: _.” This error message clearly specifies the exact position and type of the issue, helping users quickly locate the error in their code.

Through this test, it is evident that the lexical analyzer can not only accurately recognize valid lexical tokens but also effectively capture and report illegal characters through its error-handling mech-

anism. This error-reporting feature significantly enhances debugging efficiency, helping developers quickly identify and resolve syntax issues in the input code, thus ensuring the reliability and robustness of the entire analysis process.

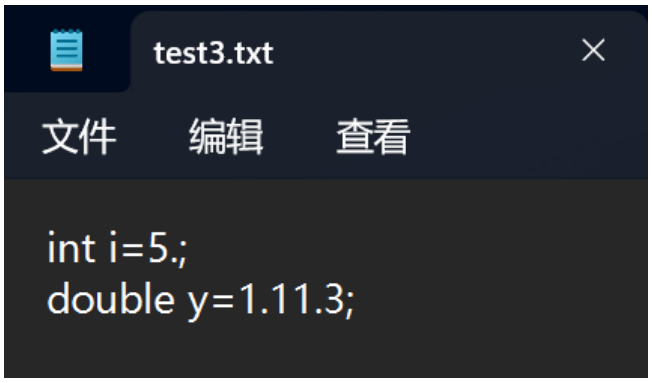

Figure 1.10 test3

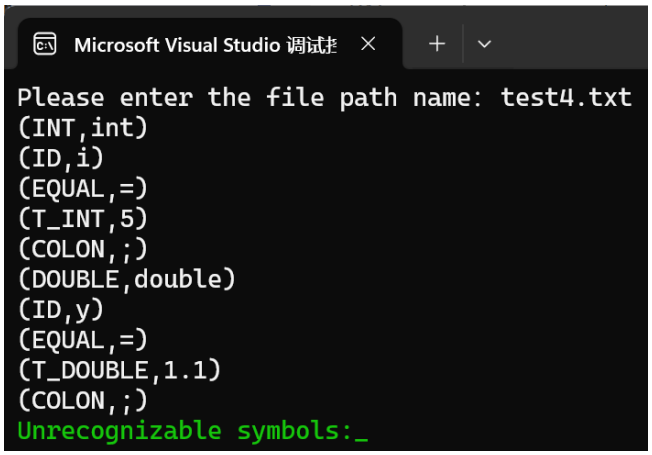

Figure 1.11 the result of rest3

1.8.4 File cannot be opened

When the program fails to open a file, the lexical analyzer outputs an exception message in the console, as shown in Figure 1.12. In this process, the user is first prompted to enter the file path to be analyzed (e.g., `test.txt`). Subsequently, the program attempts to open the specified file based on the provided path. However, if the file path is incorrect, the file does not exist, or the file cannot be accessed (e.g., due to insufficient permissions), the program returns an error message and notifies the user of the specific issue in the form of a prompt—"`Unable to open file: test.txt`."

This design aims to enhance the usability and diagnostic capability of the program, allowing users to quickly identify the issue and take appropriate corrective actions. By incorporating such an exception-handling mechanism, the program avoids runtime interruptions caused by incorrect file paths and provides a more user-friendly experience. This mechanism is particularly important as it effectively mitigates potential issues caused by input errors, thereby ensuring the program's stability and robustness.

Figure 1.12 Unable to open file output results

1.9 Problems occurred and related solutions

During this experiment, I encountered several issues in implementing the code. Below are the main problems and the analysis of their solutions:

1. How to Define the Workflow of the Lexical Analysis Program

The basic operational logic of a lexical analyzer appears straightforward at first glance. It can directly compare the input characters one by one with elements in the lexeme table to determine their types. However, this approach is inefficient. As the size of the lexeme table grows, the time complexity of this method increases to $O(n)$ (where n is the total number of lexemes), which significantly impacts the overall performance of the program.

To improve the efficiency of the lexical analyzer, I optimized the workflow. At the initial stage of lexical analysis, I used ASCII values to quickly classify characters (e.g., digits, letters) to avoid direct comparisons with the lexeme table for every character. This optimization effectively reduced the time complexity of the lookup process, laying a foundation for faster token recognition. Additionally, I incorporated principles of finite-state machines to further optimize the workflow, making the lexical analyzer more efficient in classification and recognition.

2. Issues with Recognizing Numeric Types

In the experiment, I defined two types of numeric tokens: integers and floating-point numbers. However, during the initial implementation, I encountered problems in recognizing floating-point numbers. Specifically, when a decimal number was input, the decimal point caused it to be incorrectly split into two separate integer tokens. After identifying this issue, I explored multiple solutions.

Eventually, I found that the problem could be resolved by modifying the function responsible for numeric recognition. In this function, I added logic to detect the presence of a decimal point, allowing it to recognize a single decimal point within a sequence of digits. Furthermore, the function ensured that both parts surrounding the decimal point adhered to numeric formatting requirements. With this modification, floating-point numbers could be correctly identified without being mistakenly split into two integer tokens.

Through these improvements, I gained a deeper understanding of the design and implementation of lexical analyzer workflows. Additionally, I realized that during experimentation, it is crucial not only to ensure the correctness of algorithmic logic but also to consider efficiency optimization and the handling of edge cases.

1.10 feelings and comments

This experiment allowed me to deeply explore one of the essential components of a compiler—the implementation principles of a lexical analyzer. By studying theoretical concepts such as regular expressions and finite state automata (DFA), I not only gained a deeper understanding of automata theory but also learned how to apply it in the context of lexical analysis. These theoretical foundations provided solid support for the practical implementation of the experiment.

During the experiment, I designed and implemented several key modules to realize the functionality of the lexical analyzer. By analyzing the input characters one by one, I successfully identified keywords, operators, identifiers, and other types of lexemes. In addition, I incorporated an error-handling module to effectively detect and handle non-compliant lexemes, providing detailed feedback and ensuring the stability and reliability of the analyzer under various boundary conditions. Through this process, I came to understand that lexical analysis requires not only accuracy but also strong robustness to handle complex input scenarios.

The design of data structures was another major focus of this experiment. I used arrays to store different categories of characters, enabling efficient classification and access. At the same time, enumeration types were utilized to map lexemes to their corresponding categories, allowing the program to quickly match and process them during subsequent stages. This data structure design not only improved the execution efficiency of the lexical analyzer but also provided a unified and standardized input format for the syntax analyzer, laying a solid foundation for the subsequent stages of compilation.

Overall, this experiment gave me a comprehensive understanding of the complexity and significance of designing and implementing a lexical analyzer by combining theory with practice. By solving problems encountered during the experiment, I enhanced my understanding of compiler theory, improved my skills in algorithm design and program implementation, and recognized the critical role of modular design and data structure selection in solving practical problems. These experiences will serve as a solid foundation for my future research and implementation in compiler development.

Chapter A Source Code for the Lexical Analyzer

Listing A.1 Lexical Analyzer Code

```
#include <iostream>
#include <string>
#include <Windows.h>
#include<fstream>
#include<cstring>

using namespace std;

/* 单词编码 */
char keyword[][20] = { "if","then",
    ↪ "read","write","case","continue","else","for","switch",
    ↪ "while","return","void","int","double","cin","cout","endl" }; //17个关键字
char delimiter[][10] = { "(",")","[","]","{","}",":",",",";","'","#",'"',
    ↪ };//12个界符
char operation[][10] = {
    ↪ "+","-","*","/","=",">","<","==",">=","<=","!=","&&","||","!","++",
    ↪ "--","<<",">>" };//18个运算符
int numKey = 17;
int numDel = 12;
int numOpe = 18;
string stack = "";//工作栈
int row = 1;
#define Maxline  1024

enum TokenCode
{
    // 关键字
    IF,
    THEN,
    READ,
    WRITE,
    CASE,
    CONTINUE,
    ELSE,
    FOR,
    SWITCH,
    WHILE,
```

```
    RETURN,
    K_VOID,
    K_INT,
    K_DOUBLE,
    K_CIN,
    K_COUT,
    K_ENDL,
    //运算符
    PLUS,
    MINUS,
    MUTIPLE,
    DIVIDE,
    EQUAL,
    BIGGER,
    SMALLER,
    E_COMPARE,
    BIGGER_EQUAL,
    SMALLER_EQUAL,
    NOT_EQUAL,
    AND,
    OR,
    NOT,
    SELF_PLUS,
    SELF_MINUS,
    LEFT_SHIFT,
    RIGHT_SHIFT,
    // 分隔符
    LEFT_YKH,
    RIGHT_YKH,
    LEFT_FKH,
    RIGHT_FKH,
    LEFT_DKH,
    RIGHT_DKH,
    COMMA,
    SEMOCOLOM,
    COLON,
    QUOTATION2,
    QUOTATION1,
    ZS,
    //标识符
    ID,
    // 数字
    T_INT,
    T_DOUBLE,
    //无法识别
    UNDEFINE
};
```

```
80
81  bool isLetter(char letter);      //判断是否是整数
82  bool isDigit(char digit);        //判断是否是字母
83  int  Is_keyword(string s);
    ↪ //判断字符串是否是关键字,是则返回下标, 否则返回-1
84  int  Is_delimiter(string s);      //判断字符是否是分隔符, 是返回下标, 否则返回-1
85  int  Is_calculation(string s);  //判断字符串是否是运算符, 是返回下标, 否则返回-1
86
87  bool isLetter(char letter)
88  {
89      if ((letter >= 'a' && letter <= 'z') || (letter >= 'A' && letter <= 'Z'))
90          return true;
91      return false;
92
93  }
94  bool isDigit(char digit)
95  {
96      if (digit >= '0' && digit <= '9')
97          return true;
98      return false;
99  }
100 int Is_keyword(string s) {
101     for (int i = 0; i < numKey; ++i) {
102         if (s == keyword[i]) {
103             return i;
104         }
105     }
106     return -1;
107 }
108 int Is_delimiter(string s) {
109     for (int i = 0; i < numDel; ++i) {
110         if (s == delimiter[i]) {
111             return i;
112         }
113     }
114     return -1;
115 }
116 int Is_calculation(string s) {
117     for (int i = 0; i < numOpe; ++i) {
118         if (s == operation[i]) {
119             return i;
120         }
121     }
122     return -1;
123 }
124 bool isInteger(const string& str) {
125     if (str.empty() || ((!isdigit(str[0])) && (str[0] != '-') && (str[0] !=
```

```
        ↪ '+')))
126         return false;
127
128     char* p;
129     strtol(str.c_str(), &p, 10);
130
131     return (*p == 0);
132 }
133
134 const char* getTokenCodeName(TokenCode code) {
135     switch (code) {
136     case IF: return "IF";
137     case THEN: return "THEN";
138     case READ: return "READ";
139     case WRITE: return "WRITE";
140     case CASE: return "CASE";
141     case CONTINUE: return "CONTINUE";
142     case ELSE: return "ELSE";
143     case FOR: return "FOR";
144     case SWITCH: return "SWITCH";
145     case WHILE: return "WHILE";
146     case RETURN: return "RETURN";
147     case K_VOID: return "VOID";
148     case K_INT: return "INT";
149     case K_DOUBLE: return "DOUBLE";
150     case K_CIN: return "CIN";
151     case K_COUT:return "COUT";
152     case K_ENDL: return "ENDL";
153     case PLUS: return "PLUS";
154     case MINUS: return "MINUS";
155     case MUTIPLE: return "MUTIPLE";
156     case DIVIDE: return "DIVIDE";
157     case EQUAL: return "EQUAL";
158     case BIGGER: return "BIGGER";
159     case SMALLER: return "SMALLER";
160     case E_COMPARE: return "E_COMPARE";
161     case BIGGER_EQUAL: return "BIGGER_EQUAL";
162     case SMALLER_EQUAL: return "SMALLER_EQUAL";
163     case NOT_EQUAL: return "NOT_EQUAL";
164     case AND: return "AND";
165     case OR: return "OR";
166     case NOT: return "NOT";
167     case SELF_PLUS: return "SELF_PLUS";
168     case SELF_MINUS: return "SELF_MINUS";
169     case LEFT_SHIFT: return "LEFT_SHIFT";
170     case RIGHT_SHIFT: return "RIGHT_SHIFT";
171     case LEFT_YKH: return "LEFT_YKH";
```

```
172     case RIGHT_YKH: return "RIGHT_YKH";
173     case LEFT_FKH: return "LEFT_FKH";
174     case RIGHT_FKH: return "RIGHT_FKH";
175     case LEFT_DKH: return "LEFT_DKH";
176     case RIGHT_DKH: return "RIGHT_DKH";
177     case COMMA: return "COMMA";
178     case SEMOCOLOM: return "SEMOCOLOM";
179     case COLON: return "COLON";
180     case QUOTATION2: return "QUOTATION2";
181     case QUOTATION1: return "QUOTATION1";
182     case ZS: return "ZS";
183     case ID: return "ID";
184     case T_INT: return "T_INT";
185     case T_DOUBLE: return "T_DOUBLE";
186     case UNDEFINE: return "UNDEFINE";
187     default: return "UNKNOWN";
188     }
189 }
190 void print(TokenCode code, const string& stack) {
191     const char* codeName = getTokenCodeName(code);
192     cout << '(' << codeName << ',' << stack << ")" << endl;
193 }
194 void lexicalAnalysis(const string& filePath) {
195     ifstream inFile(filePath);
196     if (!inFile) {
197         cerr << "Unable to open file: " << filePath << endl;
198         return;
199     }
200     string currentToken;
201     TokenCode currentTokenType;
202 
203     char currentChar;
204     while (inFile.get(currentChar)) {
205         if (isLetter(currentChar)) {//判断是否是ID类
206             string currentToken;
207             currentToken += currentChar;
208             char nextChar;
209             while (inFile.get(nextChar) && (isLetter(nextChar) ||
                ↪ isDigit(nextChar))) {
210                 currentToken += nextChar;
211             }
212             inFile.putback(nextChar);
213             int keywordIndex = Is_keyword(currentToken);
214             if (keywordIndex != -1) {
215                 print(static_cast<TokenCode>(keywordIndex), currentToken);
216             }
217             else {
```

```
218             print(ID, currentToken);
219         }
220     }
221     else if (isDigit(currentChar)) {//判断是否是数字类
222         string currentToken;
223         currentToken += currentChar;
224         char nextChar;
225         bool isFloat = false; //浮点数标记
226         while (inFile.get(nextChar) && (isDigit(nextChar) || (nextChar ==
            ↪ '.'))) {
227             if (nextChar == '.' && isFloat) {
228                 SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
                    ↪ FOREGROUND_INTENSITY | FOREGROUND_RED);
229                 cout << "Floating point type error:" << currentToken +
                    ↪ nextChar << endl;
230                 return;
231             }
232             if (nextChar == '.') {
233                 isFloat = true;
234                 char n_char;
235                 inFile.get(n_char);
236                 if (!isDigit(n_char)) {
237                     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
                        ↪ FOREGROUND_INTENSITY | FOREGROUND_RED);
238                     cout << "Floating point type error:" << currentToken +
                        ↪ nextChar + n_char << endl;
239                     return;
240                 }
241                 inFile.putback(n_char);
242             }
243             currentToken += nextChar;
244
245
246
247
248         }
249         inFile.putback(nextChar);
250         //判断是浮点数还是整数
251         if (isFloat) {
252             print(T_DOUBLE, currentToken);
253         }
254         else {
255             print(T_INT, currentToken);
256         }
257     }
258     else if (Is_delimiter(string(1, currentChar)) != -1 ||
        ↪ Is_calculation(string(1, currentChar)) != -1)
```

```
            ↪ {//分隔符和操作符判断
259             string currentToken;
260             currentToken += currentChar;
261 
262             char nextChar;
263             while (inFile.get(nextChar)) {
264                 string combinedToken = currentToken + nextChar;
265                 if (Is_delimiter(combinedToken) != -1 ||
                        ↪ Is_calculation(combinedToken) != -1) {
266                     currentToken = combinedToken;
267                 }
268                 else {
269                     inFile.putback(nextChar);
270                     break;
271                 }
272             }
273             int delimiterIndex = Is_delimiter(currentToken);
274             int operatorIndex = Is_calculation(currentToken);
275             if (delimiterIndex != -1) {
276                 print(static_cast<TokenCode>(numKey + numOpe + delimiterIndex),
                        ↪ currentToken);
277             }
278             else if (operatorIndex != -1) {
279                 print(static_cast<TokenCode>(numKey + operatorIndex),
                        ↪ currentToken);
280             }
281 
282         }
283         else if (currentChar == ' ' || currentChar == '\n')
284         {
285             continue;
286         }
287         else {
288             SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
                    ↪ FOREGROUND_INTENSITY | FOREGROUND_GREEN);
289             cout << "Unrecognizable symbols:" << currentChar << endl;
290             return;
291         }
292     }
293     inFile.close();
294 }
295 
296 int main() {
297     string filename;
298     cout << "Please enter the file path name: ";
299     cin >> filename;
300     lexicalAnalysis(filename);
```

```
301     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
            ↪ FOREGROUND_INTENSITY | FOREGROUND_RED | FOREGROUND_GREEN |
            ↪ FOREGROUND_BLUE);    //字体恢复原来的颜色
302     return 0;
303 }
```